Terrain classification from real robot sensor data

A real-data comparison of statistical features and temporal CNNs | Project report

Abstract

This project studies whether short windows of robot motion and wheel measurements can identify the terrain beneath a mobile robot. It uses the BorealTC dataset, with measurements from a wheeled Husky robot, and compares an RBF support vector machine (SVM), a one-dimensional convolutional neural network (CNN), and a CNN that combines learned temporal features with engineered statistics. The selected CNN plus statistics model achieved 86.3% accuracy, 0.861 balanced accuracy, and 0.845 macro-F1 on 930 held-out windows from unseen recordings. These results describe one fixed recording split and one training seed. The work adapts the research question of tactile terrain classification to different sensors and labels; it does not reproduce the original experiment.

1. Introduction

Terrain affects a robot's motion and wheel response. A classifier that uses onboard sensor time series could provide a useful estimate of the surface during operation. The project asks whether summary statistics, temporal patterns learned by a CNN, or a combination of both best distinguish five recorded terrain classes.

1.1 Problem statement

Given a 1.7-second window of inertial and wheel measurements, predict its recorded terrain label: asphalt, flooring, ice, sandy loam, or snow.

1.2 Objectives

Prepare a recording-wise train, validation, and test split; extract reproducible signal statistics; compare an RBF SVM with raw-signal and feature-fusion CNNs; report accuracy and class-balanced metrics; and provide a local interactive demo using saved project artifacts.

1.3 Project contributions

The repository combines a pinned dataset downloader with hash checks, timestamp validation and causal sensor alignment, recording-level partitioning before windowing, three model comparisons with saved predictions, and a Streamlit demo that loads the trained artifacts.

Key result: the CNN plus statistics model correctly classified 803 of 930 held-out windows.

2. Background and dataset

Research context and measured data

2.1 Research context

The project takes inspiration from Choi and Thomasson's study of terrain classification for small legged robots using tactile signals [1]. That work compares engineered features and temporal CNN representations. The present implementation preserves this broad comparison while using a different robot, different sensors, a different set of labels, and a new model configuration. It is therefore a real-data adaptation rather than an exact reproduction.

BorealTC supplies measurements from a Husky wheeled robot and provides five single-terrain labels used here [2, 3]. The original tactile study's SAIL-R data are not part of this project. The project also does not claim to reproduce BorealTC's published benchmark.

2.2 Dataset provenance

The repository pins BorealTC revision `0146dcd9fa08c34a9075a80448ac04c0a947b568`. Its manifest covers 144 files (142 sensor CSVs plus the dataset README and license), totaling 90,534,541 bytes. The attached project ZIP contains the manifest and downloader but not the raw CSVs. The pinned files were retrieved in a working copy, and the downloader verified the upstream Git blob hashes.

Terrain label	Recordings	Sensor CSVs
Asphalt	7	14
Flooring	10	20
Ice	13	26
Sandy loam	14	28
Snow	27	54
Total	71	142

2.3 Recording and window counts

Terrain	Train	Validation	Test	Total
Asphalt	275	5	51	331
Flooring	617	259	378	1,254
Ice	1,049	67	221	1,337
Sandy loam	211	83	87	381
Snow	502	157	193	852
Total	2,654	571	930	4,155

The source folder name SANDY_LOAM is preserved in the code; the dataset description calls the material silty loam. MIXED recordings are excluded because the pipeline assigns one label to each recording.

3. Data preparation and features

The pipeline validates and partitions recordings before creating model inputs

3.1 Sensor channels and alignment

Each paired recording supplies an IMU file and a wheel-proprioception file. The model uses ten channels: angular velocity (wx, wy, wz), acceleration (ax, ay, az), left and right motor current (curL, curR), and left and right wheel speed (velL, velR). The code checks required columns, finite values, and strictly increasing timestamps.

The IMU timestamps define a 100 Hz grid. Wheel measurements are aligned with the latest preceding wheel value using a backward as-of join, restricted to the shared time coverage. This causal hold avoids using a future wheel reading. No interpolation is applied.

3.2 Recording-wise split and windows

The project assigns whole recording IDs to training, validation, or test within each terrain class using seed 42. There are 43 training recordings, 14 validation recordings, and 14 test recordings. The split happens before window creation so adjacent windows from a single run cannot appear in more than one partition.

Within each recording, the pipeline advances in steps of 170 samples to make non-overlapping windows. Each window contains 10 channels by 170 time steps, or 1.7 seconds at 100 Hz. Incomplete trailing samples are discarded. Unequal recording lengths create uneven window proportions even though the recording-level allocation is roughly 60/20/20.

3.3 Statistical features and scaling

For each channel, six statistics summarize the 170 samples: mean, population standard deviation, minimum, maximum, root mean square (RMS), and first-difference RMS. The resulting vector contains 10 x 6 = 60 features. The first-difference RMS measures adjacent-sample variation; it is not scaled by the sample interval and is not a derivative in units per second.

The CNN input is normalized using per-channel means and standard deviations fitted on training windows only. Feature scaling is also fitted on training data only. Validation and test windows use these stored training transformations. This keeps held-out data out of preprocessing fit.

Processing summary

Stage	Implementation
Validate	Check paired files, required columns, finite measurements, and timestamp order.
Align	Join wheel data to the 100 Hz IMU grid with the latest preceding value.
Partition	Assign full recording IDs to train, validation, or test within terrain.
Window	Create non-overlapping 170-sample windows and discard incomplete tails.
Scale	Fit signal and feature scaling on training data and reuse for held-out data.

No synthetic training data or augmentation is used.

4. Models and evaluation

Three model families compare engineered and learned representations

4.1 Candidate models

Model	Input and design	Selection or role
Majority baseline	Always predicts the most common training class, ICE.	Reference floor.
RBF SVM	60 standardized features; balanced class weights.	Eight C/gamma combinations scored on validation macro-F1.
1D CNN	10 x 170 normalized sensor traces.	Best validation checkpoint is epoch 26.
CNN + statistics	CNN representation concatenated with the 60 feature values.	Best validation checkpoint is epoch 39.

4.2 CNN architecture

The raw-signal path uses four Conv1d blocks with 24 output channels, kernel size 5, ReLU, max-pooling, and dropout 0.3. Sequence lengths progress from 170 to 85, 42, 21, and 10. Flattening 24 channels by 10 positions produces a 240-value representation. The classifier uses a 64-unit hidden layer and five output logits. The hybrid model concatenates 60 standardized statistics to the learned representation, giving a 300-value dense input.

4.3 Training and model selection

CNN training uses Adam with learning rate 0.001, weight decay 0.0001, batch size 64, and weighted cross-entropy. Training stops after 60 epochs at most, with early-stopping patience 12. The random seed is 42, and deterministic Torch algorithms are enabled. The SVM search evaluates C in {0.1, 1, 10, 100} with gamma in {scale, 0.01}.

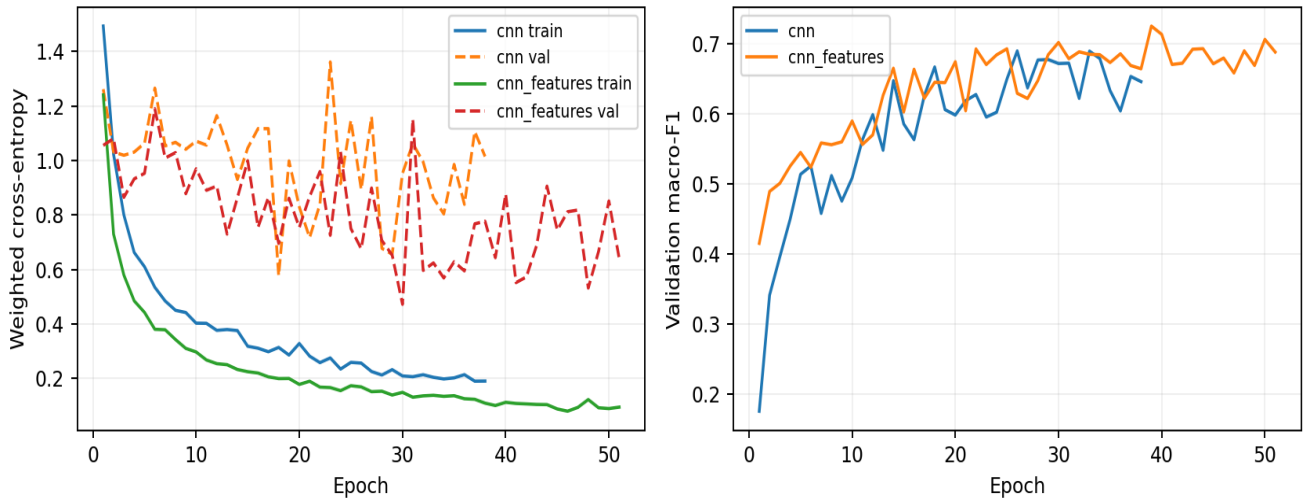
Validation macro-F1 selects the model or CNN checkpoint. The held-out test set is used for final reporting only.

4.4 Metrics

Accuracy is the fraction of correctly classified windows. Balanced accuracy averages recall across classes, so each terrain contributes equally. Macro-F1 averages each class F1 equally. For a class, precision measures the share of predicted examples that are correct; recall measures the share of its true examples recovered.

The split contains 930 test windows, but those windows come from only 14 recordings and remain correlated within a recording. Results are therefore reported as one split-level estimate, not as 930 independent field trials.

Figure 1. Saved training histories. Validation macro-F1 selects the raw CNN checkpoint at epoch 26 and the hybrid checkpoint at epoch 39.



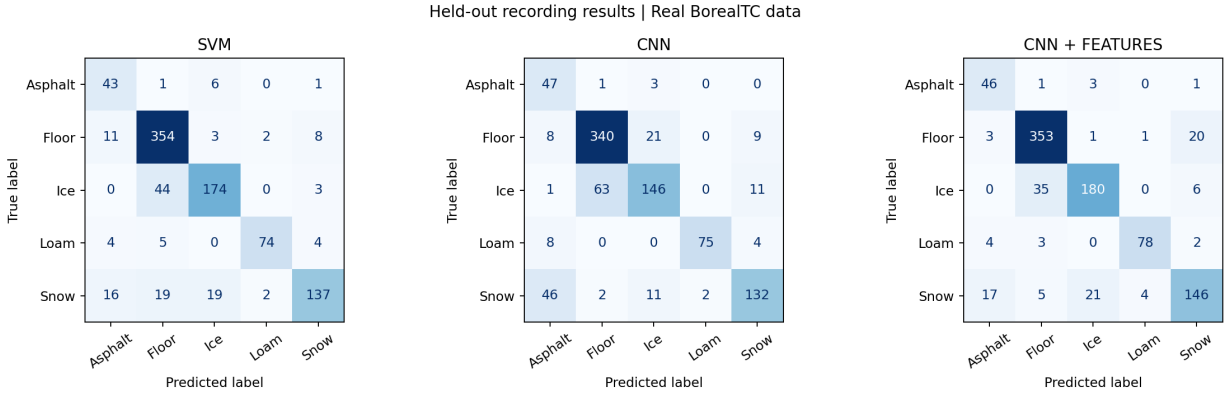
5. Results and discussion

Validation scores choose the model; test scores estimate held-out performance

Model	Val. macro-F1	Test accuracy	Balanced accuracy	Test macro-F1
Majority baseline	0.042	0.238	0.200	0.077
RBF SVM	0.688	0.841	0.825	0.817
1D CNN	0.690	0.796	0.806	0.770
CNN + statistics	0.726	0.863	0.861	0.845

The CNN plus statistics model led validation macro-F1 at 0.726 and was selected. On the test set it reached 0.863 accuracy, 0.861 balanced accuracy, and 0.845 macro-F1, correctly classifying 803 of 930 windows. Its observed accuracy was 2.3 percentage points above the SVM. One split and one seed do not establish statistical significance.

Figure 2. Test confusion matrices for the saved predictions. Rows show true labels; columns show predicted labels.



CNN plus statistics performance by terrain

Terrain	Precision	Recall	F1	Test windows
Asphalt	0.657	0.902	0.760	51
Flooring	0.889	0.934	0.911	378
Ice	0.878	0.814	0.845	221
Sandy loam	0.940	0.897	0.918	87
Snow	0.834	0.756	0.793	193

The largest off-diagonal counts were ice classified as flooring (35), snow as ice (21), flooring as snow (20), and snow as asphalt (17). Loam and flooring had the highest F1 values; snow had the lowest recall. The counts identify where errors occurred but do not explain why signals overlap.

6. System, limitations and conclusion

Implementation details and responsible interpretation

6.1 Interactive demonstration

The repository includes a Streamlit app that loads saved model artifacts and a set of 50 held-out real windows, with ten examples per terrain. A user can select a model and sample, inspect its prediction against the recorded label, and view signal traces and evaluation summaries. The demo uses the same saved artifacts used in the evaluation; the CNN softmax scores are not calibrated probabilities.

6.2 Repository and reproduction

Path	Role
terrain/data.py	Validate, align, split, window, and extract features.
terrain/models.py, terrain/train.py	Define and train the model candidates; save checkpoints and metrics.
terrain/predict.py, app.py	Load saved artifacts for prediction and the local Streamlit demo.
scripts/download_data.py, tests/	Retrieve pinned data with integrity checks; validate project behavior.

From the project root, install dependencies from requirements.txt, then run `python scripts/download_data.py`, `python -m terrain.data`, and `python -m terrain.train --epochs 60 --seed 42`. Start the demo with `python -m streamlit run app.py`. Data download requires internet access; the pinned subset is about 90.5 MB. The repository's validation record dated 8 October 2026 reports eight tests passed.

6.3 Limitations and next steps

This study evaluates one seed and one fixed recording split. Asphalt has only five validation windows, which limits confidence in model selection for that class. Windows from the same recording are correlated. The results do not establish generalization to other robots, live-control performance, or calibrated confidence. Further work could use grouped cross-validation and repeated seeds, test sensor ablations, and evaluate genuine SAIL-R tactile data.

6.4 Conclusion

On this BorealTC split, combining temporal CNN features with 60 engineered statistics produced the strongest validation macro-F1 and the best held-out scores among the tested candidates. The experiment supports feature fusion as a promising approach for these recorded sensor signals, while its fixed split and recording-level scope limit broader claims.

References

[1] H. Choi and R. Thomasson. *Terrain Classification for Small Legged Robots Using Deep Learning on Tactile Data.* Stanford CS229 project report, 2020. https://cs229.stanford.edu/proj2020spr/report/Choi_Thomasson.pdf
[2] D. LaRocque et al. *Proprioception Is All You Need: Terrain Classification for Boreal Forests.* IROS 2024, pp. 11686-11693. doi:10.1109/IROS58592.2024.10801407.
[3] BorealTC dataset, revision 0146dcd9fa08c34a9075a80448ac04c0a947b568. https://github.com/norlab-ulaval/BorealTC. Dataset README declares CC0 1.0.